COMPONENTE CURRICULAR:	Comunicação de Dados
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

Questão 1: Cliente-Servidor vs. Peer-to-Peer (P2P)

- **Cliente-Servidor:** Existe um servidor central (host sempre ligado) que atende requisições de vários clientes.
    - **Vantagens:** Gerenciamento centralizado e segurança facilitada.
    - **Limitações:** O servidor é um ponto único de falha e pode se tornar um gargalo (problema de escalabilidade vertical).
- **P2P (Peer-to-Peer):** Não há servidor fixo. Cada nó (par) age tanto como cliente quanto como servidor, compartilhando recursos diretamente.
    - **Vantagens:** Auto-escalabilidade (mais usuários trazem mais recursos).
    - **Limitações:** Difícil gerenciamento, segurança complexa e os nós são intermitentes (podem desconectar a qualquer momento).

Questão 2: Escalabilidade nas Arquiteturas

No modelo **Cliente-Servidor**, a escalabilidade é limitada pela capacidade do servidor central; conforme as requisições crescem, a carga no servidor aumenta, podendo causar lentidão. Já no **P2P**, a arquitetura possui **auto-escalabilidade**: cada novo par que entra na rede não apenas consome dados, mas também distribui, aumentando a capacidade total do sistema. Por isso, no P2P, o tempo de resposta tende a ser mais estável mesmo com o aumento da rede.

Questão 3: Características do Protocolo HTTP

O HTTP (*Hypertext Transfer Protocol*) opera na **Camada de Aplicação** e utiliza o **TCP** como protocolo de transporte (porta 80 ou 443). Ele utiliza a arquitetura **Cliente-Servidor**. Uma característica crucial é que ele é um protocolo **stateless** (sem estado), ou seja, o servidor não guarda informações sobre requisições passadas nativamente, tratando cada interação como se fosse a primeira.

Questão 4: Métodos HTTP (Verbos)

O campo **método** indica a ação que o cliente deseja realizar no servidor. Os três principais são:

1. **GET:** Utilizado para solicitar (buscar) dados do servidor sem alterá-los.
2. **POST:** Envia dados para o servidor processar (como um formulário ou criação de recurso).
3. **PUT:** Usado para atualizar informações existentes ou substituir um recurso no servidor.

Questão 5: Cookies

Cookies são pequenos arquivos de texto enviados pelo servidor e **armazenados no navegador do cliente**.

- **Funcionamento:** Quando o servidor responde a uma requisição, ele envia o cabeçalho Set-Cookie. O navegador armazena isso e, nas próximas requisições, envia o cookie de volta.
- **Importância:** Eles resolvem o problema do HTTP ser *stateless*, permitindo manter sessões de login, carrinhos de compras e preferências de personalização.

Questão 6: Web Cache (Servidor Proxy)

O **Web Cache** armazena cópias de objetos acessados recentemente em um servidor local da universidade.

- **Impacto/Benefícios:** Reduz o tráfego no link de saída da instituição, diminui o tempo de resposta para o usuário e economiza custos de largura de banda.
- **Atualização:** O HTTP garante a atualização através do **GET Condicional**. O cache envia uma requisição ao servidor de origem com o cabeçalho If-Modified-Since. Se o objeto não mudou, o servidor responde com um código 304 Not Modified, e o cache entrega a cópia local com segurança.

Questão 7: Envio e Recebimento de E-mail

O processo de e-mail utiliza agentes de usuário, servidores de correio e protocolos distintos:

- **Envio (SMTP):** O cliente envia o e-mail para o seu servidor, e esse servidor transfere a mensagem para o servidor do destinatário. O **SMTP** é um protocolo de "empurrar" (push).
- **Recebimento (IMAP/POP3):** O destinatário usa o **IMAP** para acessar as mensagens. O IMAP permite visualizar e organizar as mensagens mantendo-as no servidor (sincronização), ao contrário do POP3, que tende a baixar e remover do servidor.

Questão 8: DNS (Domain Name System)

O DNS traduz nomes de domínio (como mackenzie.br) em endereços IP.

- **Protocolo:** Utiliza majoritariamente o **UDP** (porta 53) pela velocidade, mas pode usar TCP para transferências de zona.
- **Arquitetura:** É um banco de dados **distribuído e hierárquico** (Root, TLD e Autoritativo), o que permite a distribuição de carga global.
- **Cache DNS:** Melhora a performance ao armazenar traduções recentes localmente (no PC ou no servidor do ISP), evitando que cada acesso precise percorrer toda a hierarquia da internet novamente.